



DEPARTMENT OF INFORMATION TECHNOLOGY
LAB MANUAL

CS23432 – Software Construction

(REGULATION 2023)

RAJALAKSHMI ENGINEERING COLLEGE

Thandalam, Chennai-602015

Name: S. DIVYA

Register No: 2116241001056

Year / Branch / Section: 2nd & Btech IT - 'A'

Semester: IV

Academic Year: 2025-2026

INDEX

S.No.	Date	Title
1.	21/1/26	Azure Devops Environment Setup.
2.	28/1/26	Azure Devops Project Setup and User Story Management.
3.	11/2/26	Setting Up Epics, Features, And User Stories for Project Planning.
4.	25/2/26	Sprint Planning.
5.	11/3/26	Poker Estimation.
6.	18/3/26	Designing Class and Sequence Diagrams for Project Architecture.
7.	25/3/26	Designing Architectural and ER Diagrams for Project Structure.
8.	1/4/26	Testing – Test Plans and Test Cases.
9.		Load Testing and Pipelines.
10.		GitHub: Project Structure & Naming Conventions.



RAJALAKSHMI ENGINEERING
COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name:S. DIVYA.....

Academic Year: ~~2020-21~~ 2021-22 Semester: IV Branch: IT

Register No

2116241001053

Certified that this is the bonafide record of work done by the above student in
the CS 23432 Software Construction Laboratory

during the academic year 2021 - 2022

Signature of Faculty in-charge

Submitted for the Practical Examination held on 12/5/22

Internal Examiner

External Examiner

I N D E X

NAME: S. Divya ROLL NO.: 241001056
STD.: _____ DIV./SEC.: _____ SUBJECT: Software Construction

S. No.	Date	Title	Page No.	Teacher's Sign/Remarks
1.	21/1/26	Azure Devops Environment Setup.	1	M. S.
2.	28/1/26	Azure Devops project Setup & User story Management	3	M. S.
3.	11/2/26	Setting up Epics, Features & User Stories for Project Planning	7	M. S.
4.	25/2/26	Sprint planning	11	M. S.
5.	11/3/26	Points Estimation	15	M. S.
6.	18/3/26	Designing class & Sequence Diagrams for Project Architecture.	19	M. S.
7.	25/3/26	Designing Architectural & ER Diagrams for Project Structure.	23	M. S.
8.	11/4/26	Testing - Test Planes & Test cases	27	
9.		Load Testing & Pipelines	30	
10.		GitLab Project Structure & Naming Conventions.	33	

11/01/26

Setting Up Envs, Features, And User Stories for Project Planning.

Exp no: 1

Date: 21/11/26

Azure DevOps Environment Setup

Aim:

To set up and access the Azure DevOps environment by creating an organization through the Azure portal.

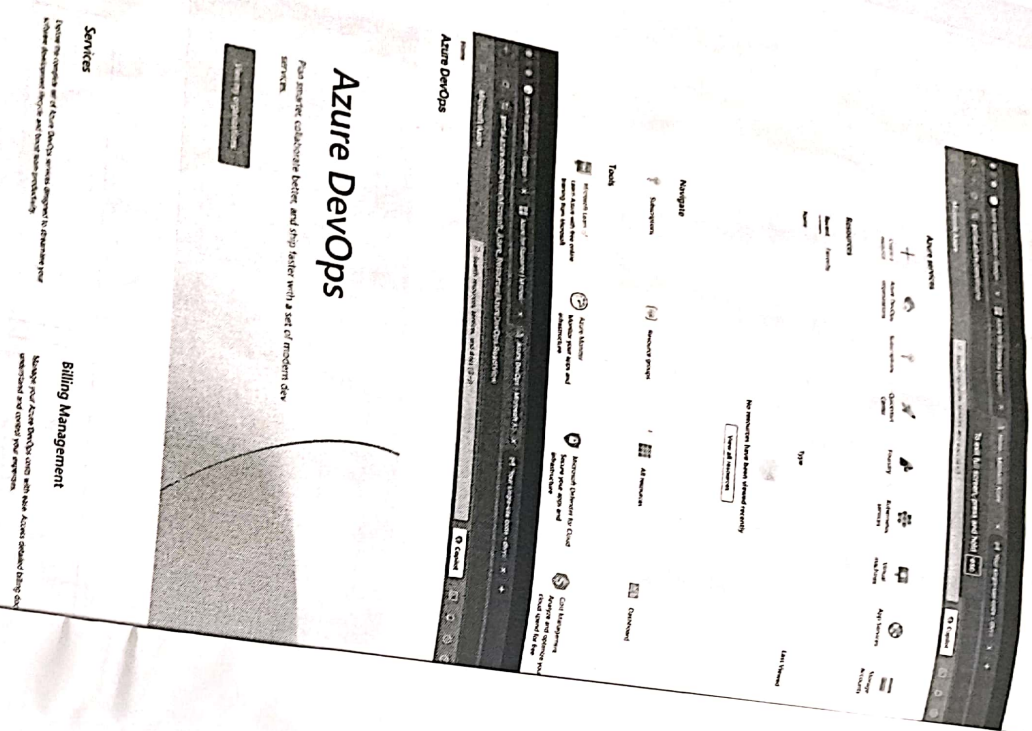
Installation:

1. Open your web browser & go to the Azure website: <https://azure.microsoft.com/en-us/get-started/azure-portal/>
- sign in using your Microsoft account credentials.
- If you don't have a Microsoft account, you can create one here: <https://signup.live.com/?lic=1>.

2. Azure home page.
3. Open DevOps environment in the Azure platform by typing Azure DevOps Organization in the search bar.
4. Click on the my Azure DevOps organization link & create an organization & you should be taken to the Azure DevOps organization home page.

Result:

Successfully accessed the Azure DevOps environment & created a new organization through the Azure portal.



Exp no: 2 Azure DevOps Project Setup and
 Date: 28/11/26 Uses sprints management.

Aim:
 To set up an Azure DevOps project for efficient collaboration & agile work management.

1. Create an Azure account.

2. Create the first project in your organisation

a. After the organisation is set up, you "ll need to create your first project. This is where you'll begin to manage code, pipelines, work items & more.

b. On the organisation's home page, click on the New project button

c. Enter your project name, description & visibility option:

Name: Todo List app to setup reminders

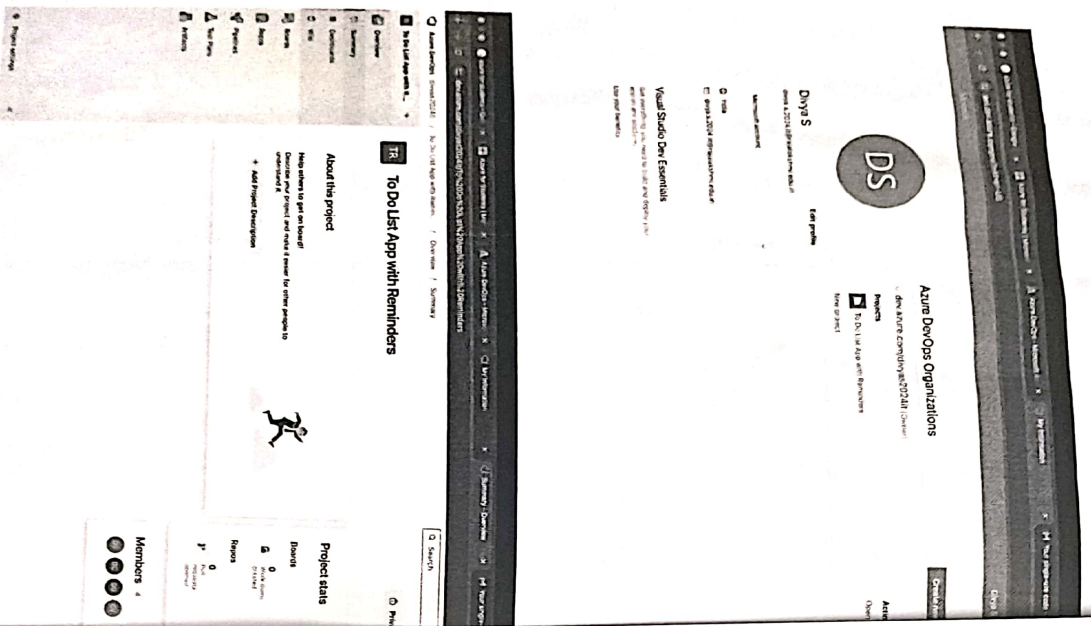
Description: Optionally, add a description to provide more context about the project

Visibility: choose whether you want the project to be private (accessible only to those invited) or public (accessible to anyone).

d. Once you have filled out the details, click Create to set up your first project.

3. Once logged in, ensure you are in the correct organization. If you're part of multiple organisations you can switch between them from the top left corner (next to your user profile). Click on the organisation name, & you should be taken to the Azure DevOps Organisation Home page.

4. Project Dashboard.



5. To manage user stories:

- a. From the left-hand navigation menu, click on Boards. This will take you to the main Boards page, where you can manage work items, backlogs, sprints.
- b. On the work item page, you'll see the option to Add a work item at the top. Alternatively, you can find a + button or Add new work item depending on the view you're in. From the Add a work item dropdown, select User story. This will open a form to enter for the new user story.

Result:

Successfully created an Azure DevOps project with user story management & agile workflow setup.

Exp no: 3

Setting up Epur, Features & User
 Date: 11/11/2020
 To-Do List / Reminders App

Aim:

To learn how to create epur, features, user stories & tasks for a To-Do List / Reminders Application

Prerequisites:

Basic Epur, Features, User Stories, Tasks

User Story (User Ideas):

As a user, I want to manage my daily tasks & set reminders so that I can stay organized & never miss important work

1. Epur in Epur:

User Epic:

→ Task Management & Reminders System

2. Epur in Features

1. > Add new tasks

2. > Edit / Update tasks

3. > Delete tasks

4. > Set reminders for tasks

5. > Mark tasks as completed

6. > Due Date & time selection

7. > Notifications & alerts

8. > User-friendly interface

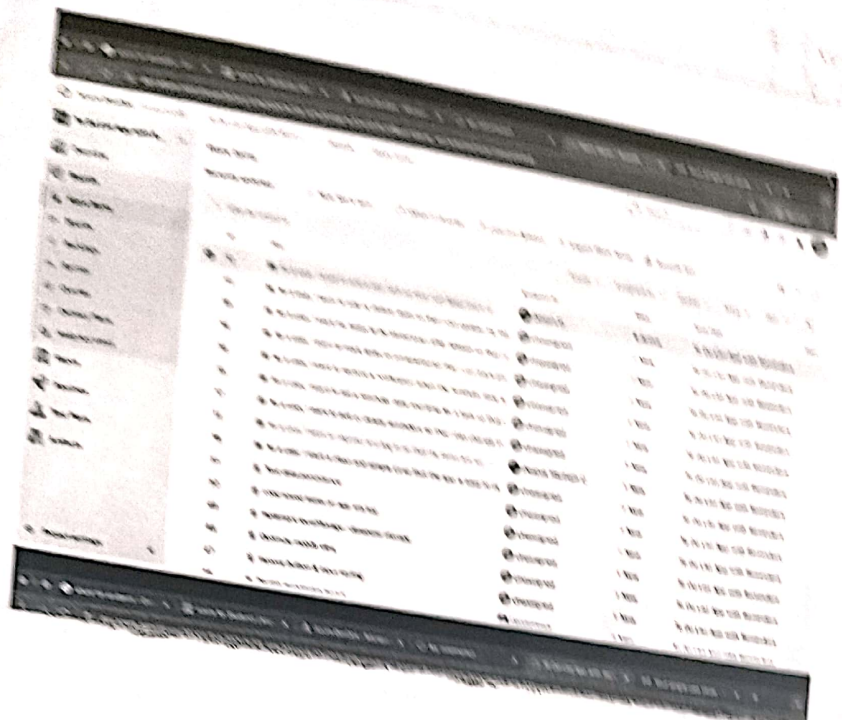
3. Epur in User Story Details

8. [E] View list of all tasks

9. [E] Add a new task with title & description

10. [E] Set reminders for a task

11. [E] Mark task as completed



1. I want to edit a task
2. I want to receive notification alerts
3. I want to delete or edit a task

4. Detailed user stories
5. As a user, I want to view all my tasks so that I can track what needs to be done.
6. As a user, I want to add a new task with details so that I can organize my work.
7. As a user, I want to set reminders for tasks so that I don't forget important activities.

8. As a user, I want to mark tasks as completed so that I can track my progress.
9. As a user, I want to edit or delete tasks so that I can manage changes.
10. As a user, I want to receive notifications so that I am alerted on time.
11. As a user, I want to set reminders for tasks so that I don't forget important activities.
12. As a user, I want to mark tasks as completed so that I can track my progress.
13. As a user, I want to edit or delete tasks so that I can manage changes.

Result:
The creation of epic, features, user stories, & tasks for the To do list / Reminder application has been completed successfully.

Exp No: 4

Sprint Planning

Date: 05/12/26

Aim:

To assign user stories to specific sprints for the To-Do list / Remainder App project.

Sprint 1:

Task board

1. View all tasks

Assigned to: Divya

Iteration 1

9. Add new task (title & description)

Assigned to: Charan

Sprint 2:

Task board

Iteration 2

10. Set remainder for tasks

Assigned to: Charan

11. Mark task as completed

Assigned to: Charan

~~Sprint 3:~~

~~Task board~~

~~Iteration 3~~

~~12. Edit / delete task~~

~~Assigned to: Divya~~

~~13. Receive notification alerts~~

~~Assigned to: Divya~~

~~14. Display user-friendly interface~~

~~Assigned to: Deepak~~

Result:

~~The sprints are created for the To-Do list / Remainder App project successfully.~~

Exp no: 5
 Date: 11/3/26

Polen Estimation:

Aim:

To use Polen estimation for the user stories of the To-Do list / Reminders App project.

Polen Estimation:

Story point measure:

→ Effort

→ complexity

→ time

We use Fibonacci scale:

1, 2, 3, 5, 8, 13, ...

Story points for To-Do list / Reminders App

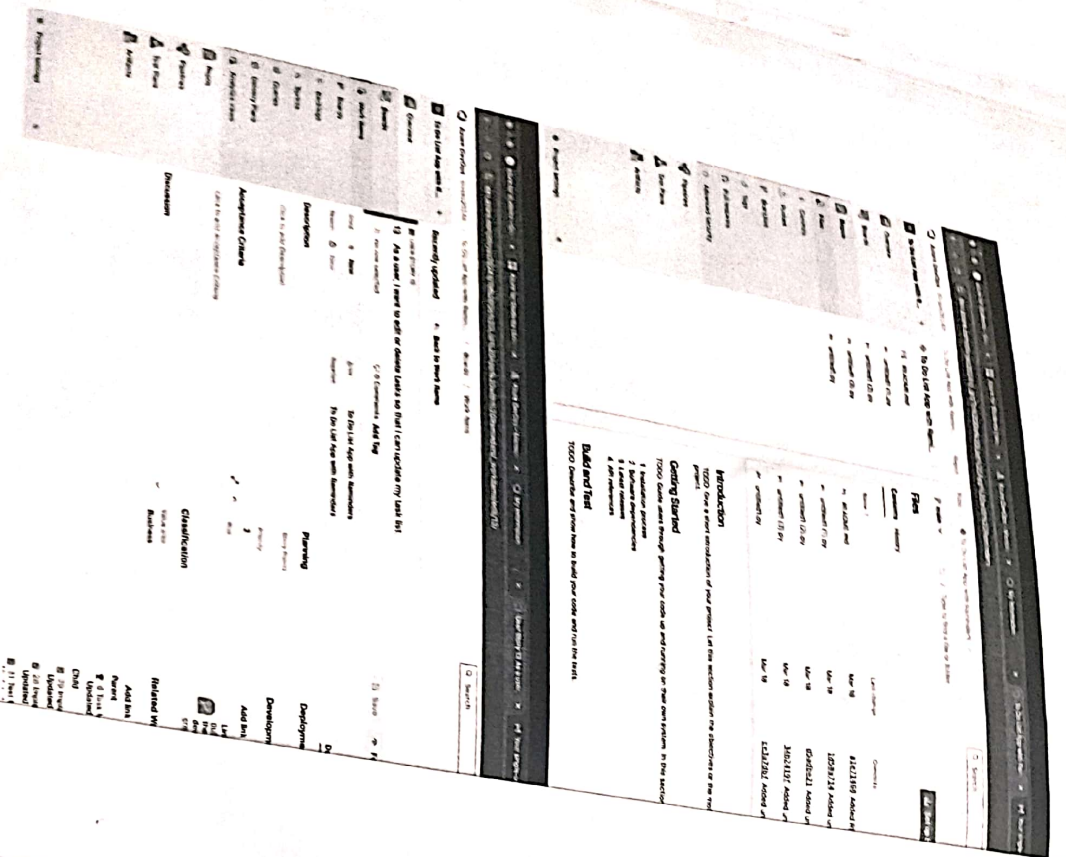
User story	Story Points	Reason
Add new task	3	Input handling + basic logic
View all tasks	2	Simple data display
Set reminder for task	5	Time scheduling + alerts
Mark task as complete	2	Simple status updates
Edit + delete task	3	CRUD operations
Receive notification alerts	5	Notification system
Display user friendly interface	3	UI design + styling

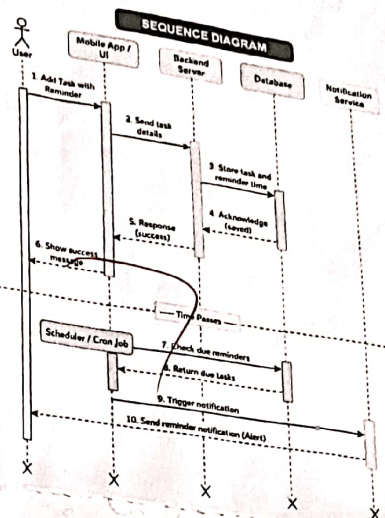
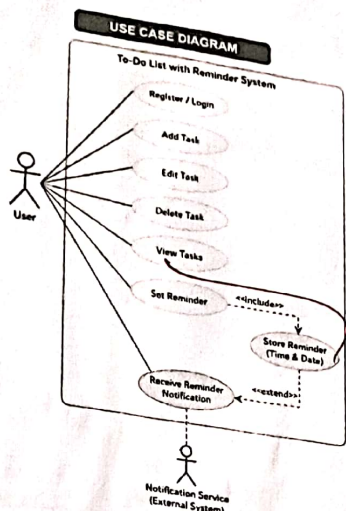
Total = 23 story points

Steps:

Breakdown → work items → user story → story points
 Repeat for all user stories.

Result: The estimation / story points are used for the To-Do list / Reminders App project using





Expno: 6 Designing class and sequence
 Date: 18/10/2023 Diagram for Project Assessment

Aim: To design a class Diagram and sequence Diagram for the To-do list with reminder application.

Objective: The goal of this experiment is to understand how different components of the application interact and how data flows between them using UML diagrams.

6.1. Class Diagram

Classes:

- User: userID, name, email
- Task: taskID, title, dueDate, status
- Reminder: reminderID, time
- Notification: message

Relationships:

- User → Task (1: N)
- Task → Reminder
- Reminder → Notification

6.2. Sequence Diagram:

Flow:

1. User enters task
2. Task saved in system
3. User sets Reminder
4. System schedules reminder
5. Notification sent at correct time

Spec, Features, User stories

Epic:

- task management app with reminders

Features:

→ Add / Edit / Delete tasks

→ Set reminders

→ Notifications

User stories:

→ User can create tasks

→ User can set reminders

→ User gets notification

Result:

~~Sequence Diagrams~~ in Sequence Diagrams were successfully designed.

Sep no: 7
25/8/20

Designing Architectural & ER Diagrams
for Project Structure.

Aim:

To design an Architectural Diagram and ER Diagram for the To-do list with Reminders App.

7A. Architectural Diagram

Explanation:

The Architectural Diagram represents the structure of the To-do list Application. It includes Frontend (UI) where user can add, edit, delete tasks & set reminders. Backend server: handles user requests & provides task data. Database: stores user details, tasks & reminders. Notification service: sends reminders to users at the scheduled time.

This diagram shows how data flows between user, server & database for smooth functioning of the app.

7B. ER Diagram

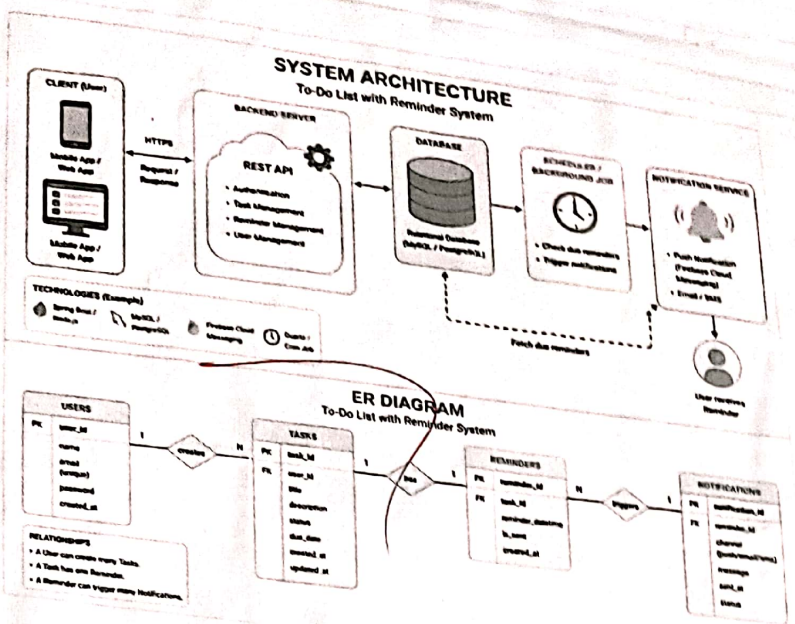
Explanation:

The ER Diagram represents the Database design of the To-do list app.

User Entity: stores user information (User ID, Name, Email)

Task Entity: stores task details (Task ID, Title, Description, Due Date, Status)

Reminder Entity: stores reminder details (Reminder ID, Time, Notification status)



Relationship:

- A user can have multiple tasks
- Each task can have one or more reminders
- This helps in organizing & managing data efficiently.

Project Structure: (Agile)

- Epic: Develop a To Do List App with reminder functionality
 - Feature: Add, Update, delete tasks, set reminders, notification system
 - User stories:
 - User can create a task
 - User can edit reminders for tasks
 - User gets notified on time.
- ⇒ Tasks:
- ⇒ Design UI
 - ⇒ Setup backend
 - ⇒ Connect database
 - ⇒ Implement reminder system.

Result:

Task, An Android & EE Diagram for the To Do List with reminder App was successfully designed.

Exp No: 8 Testing - Test plan & Test cases

Date: 11/10/20

Aim:

- To create test plans & test cases for the todo-list application with reminder functionality
- & validate system behaviour using happy path & error scenarios.

Test plan & Test case:

Test case Design Procedure:

1. Identify the main features of the system
(Task Management, Reminder setting, Notification, Task completion)
2. Define use cases
3. Create happy path test cases
4. Create error path test cases
5. Specify steps & expected results
6. Assign test case IDs

Use stories:

- user can add a task
- user can update or delete a task
- user can set a reminder for a task
- user can receive reminder notification
- user can mark task as completed

Test case:

Test Suite: Task Management

Test 1: Add Task (Happy Path)

- Action: Enter valid task details & save
- Expected Result: Task added successfully
- Test 2: Add Task (Error Path)

Test Suite: Reminder Setup

TC03 - Set reminder (Happy Path)

- Action: Set valid date & time for reminder
- Expected Result: Reminder set successfully

TC04 - Set reminder (Error Path)

- Action: Set past time / date
- Expected Result: Error msg displayed

Test Suite: Notification

TC05 - Receive Reminder (Happy Path)

- Action: wait for scheduled reminder time
- Expected Result: Notification received

TC06 - Receive Reminder (Error Path)

- Action: Disable notifications
- Expected Result: Reminder not shown / warning displayed

Test Suite: Task completion

TC07 - Mark task completed (Happy Path)

- Action: tick mark as completed
- Expected Result: task mark as completed

TC08 - Mark Task complete (Error Path)

- Action: Try marking non-existing task
- Expected Result: Error msg display

Test Suite: Delete Task

TC09 - Delete Task (Happy Path)

- Action: select task & delete
- Expected Result: task deleted successfully

TC10: Delete Task (Error Path)

- Action - Delete already deleted task.
- Expected result: Error msg displayed.

Result:

~~The test plan~~ In test plan, test cases are created successfully
~~the~~ The To Do list application with reminder functionality, considering both happy path & error scenario.

11/2/26

Setting Up Topics, Features, And User Stories for Project Planning.

Ex no: 9
Date:

Load Testing and Pipelines As, Todo list to setup Reminders

Aim:

To perform load testing on a To-Do list app with reminder feature & implement a pipeline for automation

Load Testing - Description:

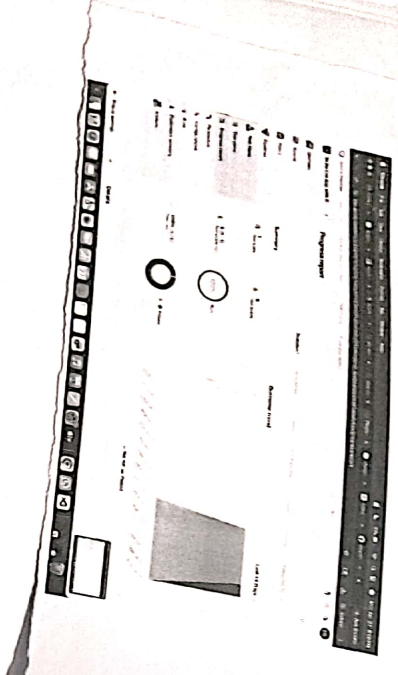
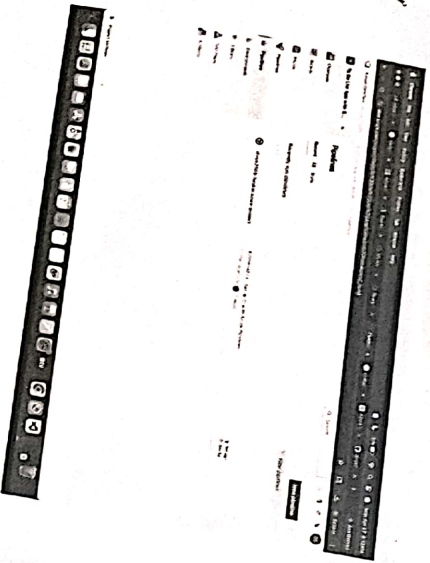
Simulate multiple user performing activities like adding tasks & setting reminders to check performance & scalability.

Steps:

1. Login to AWS Portal
2. Create AWS Load Testing Resource
3. Enter details (Subscription, Resource Group, Location)
4. Open resource → Create Test
5. Add HTTP request (Task & Reminders API)

Observation:

- Handle multiple users efficiently
- Response time stable
- Reminders triggered correctly
- No major errors
- Good scalability



Pipeline - Description:
Automate build & testing of the To-Do app using AWS DevOps

Steps:

1. Clone Github Repo
2. Create Pipeline
3. Add YAML File

</> YAML

+ Hugo;

- main

pool;

vmImage: ubuntu - latest

Steps:

- checkout: self

- tool: Use Python Versions

input:

VersionSpec: '3.x'

- script: pip install -r requirements.txt

- script: python -c "print ('Pipeline Running!')"

Observation:

- Dependencies installed successfully
- Pipelines executed without error
- Logs verified execution
- Continuous integration achieved

Result:

Successful pipeline setup for To-Do app with remaining work completed successfully with good performance & automation

Sapno:io

GitHub: Project structure & Naming conventions

Aim:

To provide a clear & organized view of the project; table structure & file naming conventions, helping contributors & users early understand, navigate & extend the code list app to setup reminders.

GitHub Project structure:

- Sign up with GitHub:
- Go to setting
- Select repositories
- Click on create new Repository
- Give name to the repository
- Now make it public
- Upload the folder from the system
- Then commit the changes.

GitHub Project structure screenshots has been posted on the left hand side.

Result:

The GitHub repository clearly displays the organized structure & consistent naming convention, making it easy for users to contribute to understand & navigate the code base.

